ROBUSTNESS OF MACHINE LEARNING MODELS UNDER DATA POISONING

Shayna Rothstein and Reshi Prager

THE CHALLENGE

- Real world data is often noisy
- Beyond that Data poisoning: malicious
- Important to build LLMs that are robust to noise and adversarial attacks

WHY THIS MATTERS- REAL WORLD APPLICATIONS

- Autonomous driving
- Medical diagnoses
- Security

DEFENSIVE VS OFFENSIVE ATTACKERS

- Defensive: maintain accuracy despite attacks
- Offensive: the attacker wants to be unnoticed
- This project focuses on the defensive side.

IDEA OF RANDOM SUBSPACES

- Concept of Random Subspace Projection
- Application for Dimensionality Reduction
- Ensemble Approach with Majority Voting
- Random orthogonal base
- Benefit Against Poisoning

EXPERIMENT/WHAT WE LOOKED AT

- **Dataset**: MNIST
- **Base models:** Linear Support Vector Machine (SVM), Logistic Regression (LogReg)
- **Label corruption**: [0.0, 0.1, 0.2, 0.3, 0.4]
- **Hyperparameters** (tuned with brute force methods):
  - Subspace dimensions k (dimensionality of the projected space),: [2, 5, 10, 20, 50]
  - Regulation Strength C (controls the model's complexity): [0.1, 1.0]

Ensemble configuration:

  - N_runs = 5

- **Training subset size**: 500
- **Evaluation metrics**
  - Test and training accuracy
  - Train-test gap: train_acc - test_acc
  - Variance

TESTING ACCURACY VS TRAINING ACCURACY

Training accuracy = How well the model performs on the data it was trained on.

Testing accuracy = How well the model performs on new and unseen data.

MODELS USED

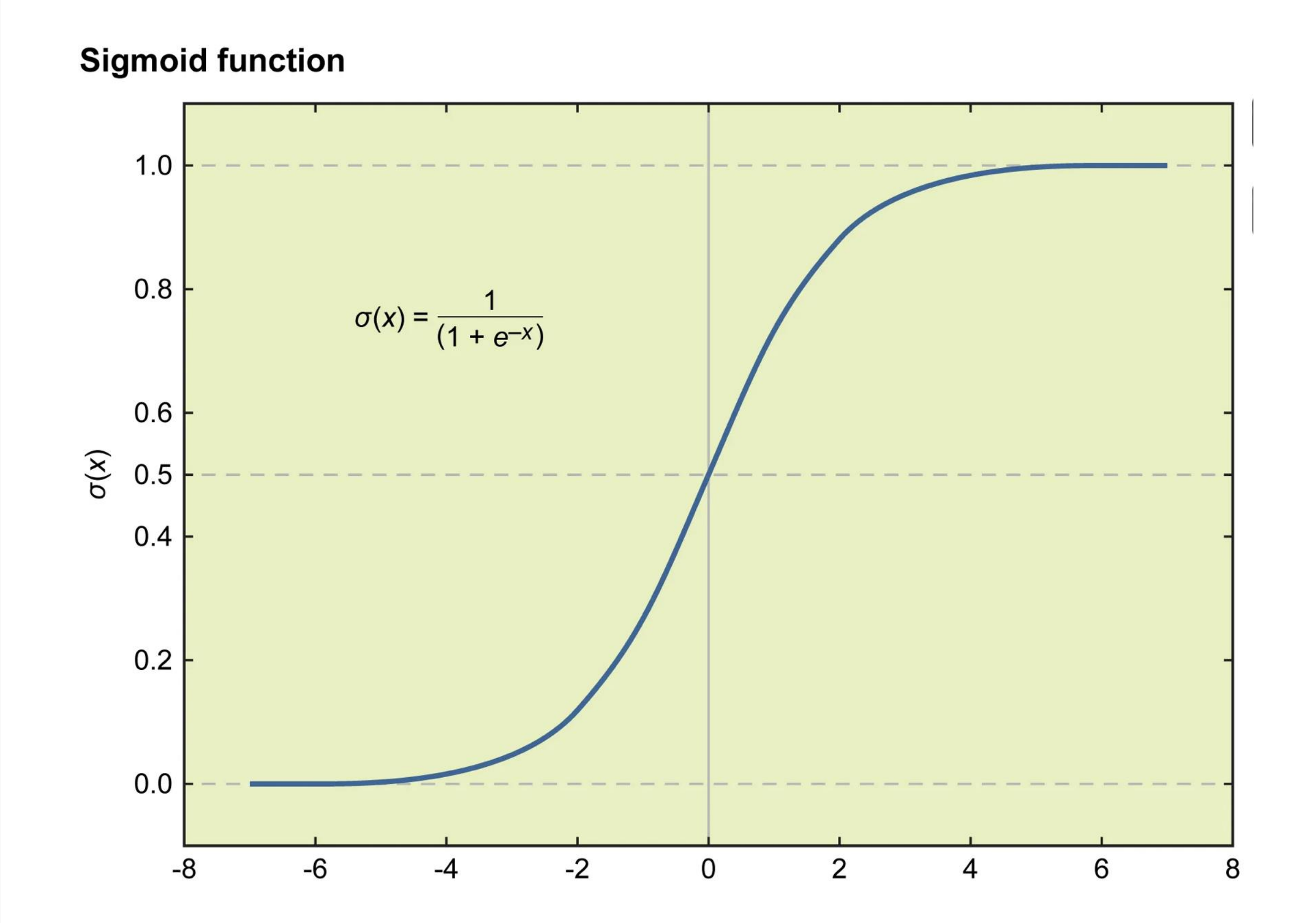
Logistic Regression and Linear SVM

LOGISTIC REGRESSION

- Logistic Regression is a binary classifier.
- Sigmoid functions: (turn into probability)

$$\sigma(z \cdot x + z_0) = \frac{1}{1+e^{-(z \cdot x + z_0)}}$$

- Learns the best weights and biases from data



LOGISTIC REGRESSION USES A LINEAR COMBINATION OF FEATURES AND A SIGMOID ACTIVATION TO ESTIMATE CLASS PROBABILITIES.

LINEAR SVM

- Linear Support Vector Machine (SVM) is also a linear classifier.
- Works by trying to find a line or hyperplane to separate two classes (in our case whether a digit is a 3 or 8 and 1 or 6)
- The support vectors are the closest points (one from each class) to the separating boundary.
- The model measures the perpendicular distance from these points to the separating line, and the smallest distance is called the margin.
- Then the SVM model tries to find the best separating line where the margin (breathing room) is as large as possible.

ROLE OF K: SUBSPACE DIMENSION

- k is the dimension of the random subspace onto which the 784 dimensional image is projected.
- Small k: Low dimension so model is less complex (line vs. hyperplane). However, because they are simpler, they are less likely to overfit to noise.
- Large k: looks at more information.

Can make more complex boundaries.

However, has more risk of overfitting (will discuss idea of overfitting more later in the slides).

ROLE OF C: A REGULARIZATION PARAMETER

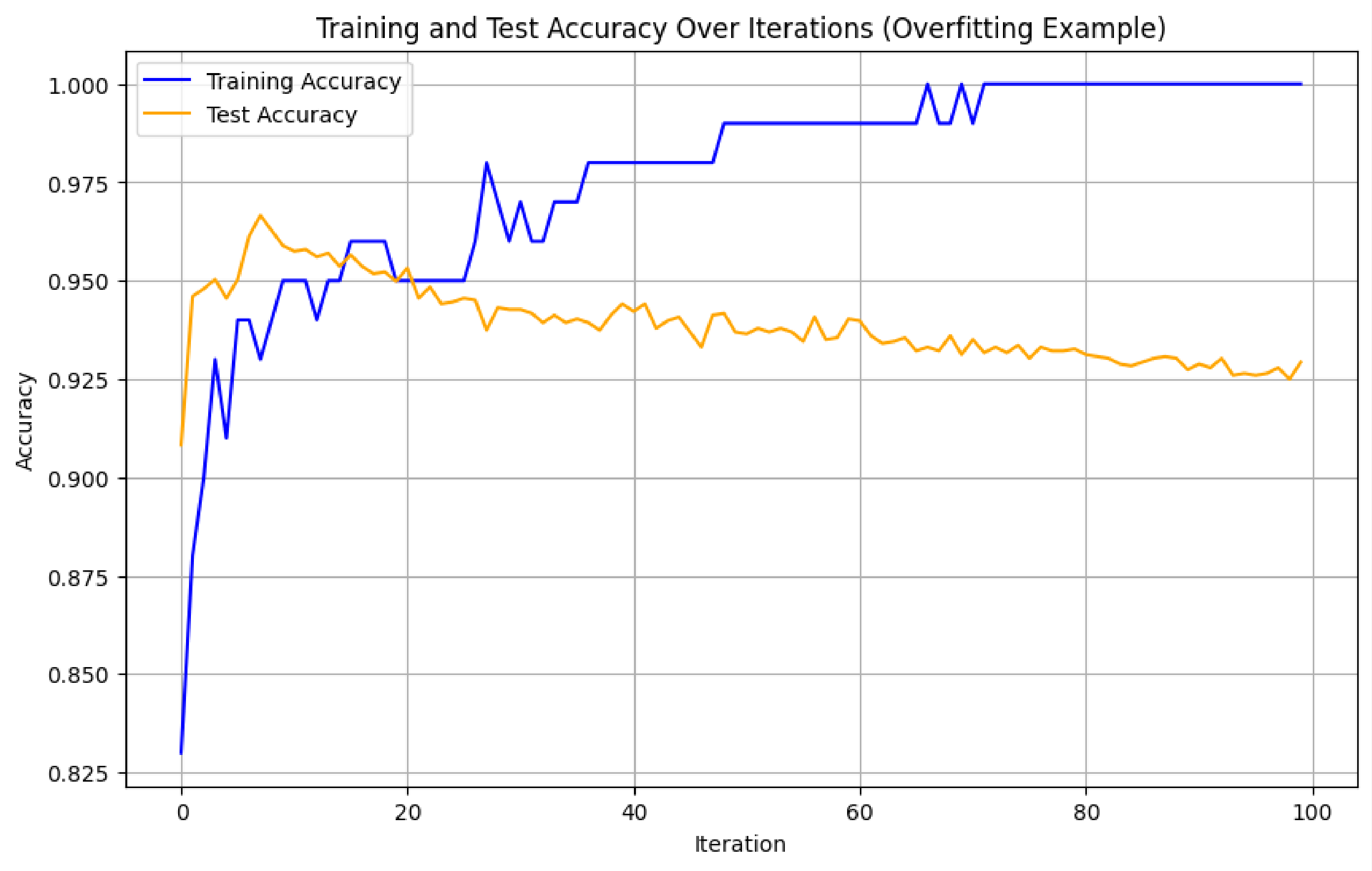
- Hyperparameter
- Regularization is used to prevent overfitting.
- The value of C is the inverse of the regularization strength, so a smaller C denotes strong regularization, while less regularization would be associated with a higher C.
- In our experiments we use different values of C to observe how regularization affects the performance of our models and how well they can detect corruption.

"RANDOM SUBSPACE METHOD" ENSEMBLE

- Each model is trained on a number of different randomly generated k dimensional subspaces.
- In our code this is represented by n_models = 5
- For each data point n_models classifies it as either a 0 or a 1.
- If more than half of the n_models classified the point as 1, the final output would be one. Otherwise, it would be 0.
- This technique can reduce overfitting and increase accuracy
- Note: n_models is different than n_runs which represents how many times we repeat the entire ensemble experiment.

OVERFITTING

- Overfitting happens when the model memorizes the training data too well- including the noise/corruption.
- This affects the performance of the model with the testing data.
- Comparison- memorize a practice test but perform poorly on the actual test.
- For a lower k, the model does not overfit as much. More simple and can not be memorized as easily.

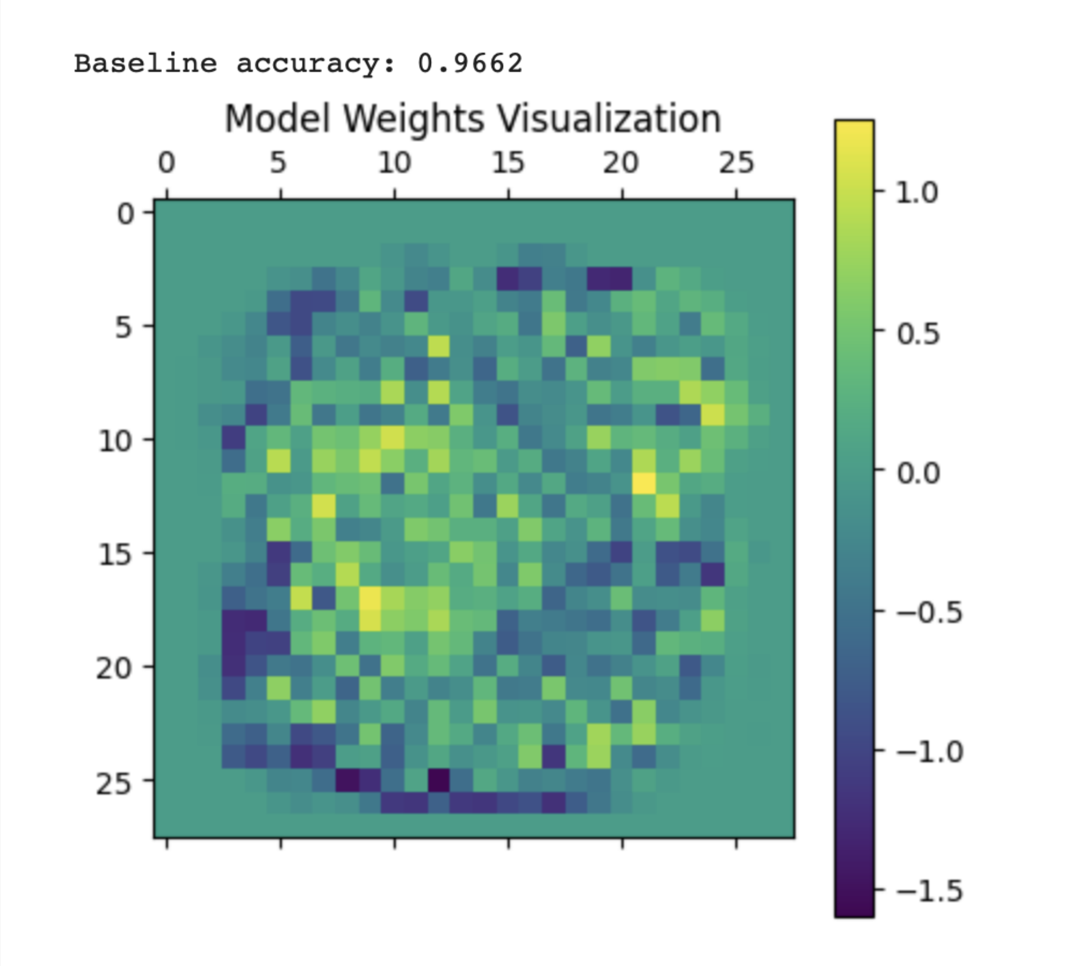


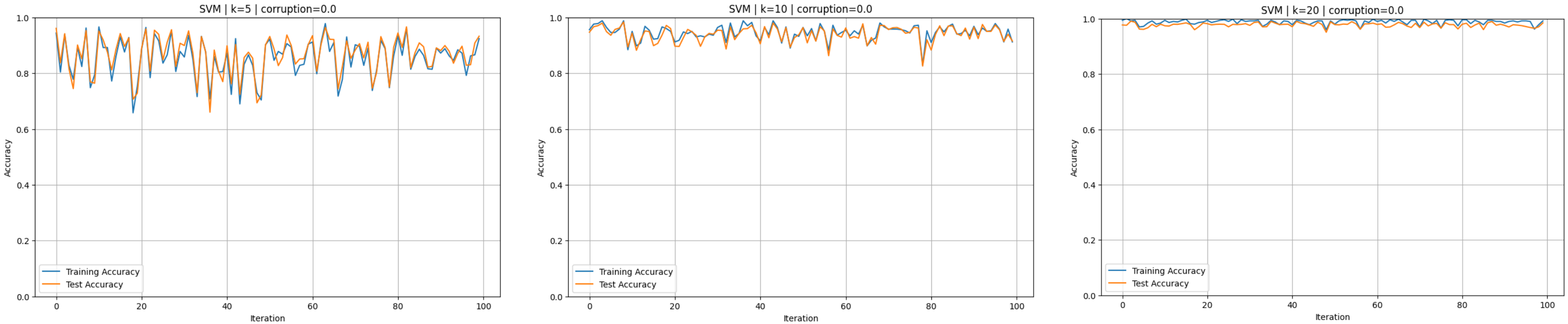
d1=1, d2=6, k=10, iterations=100, technique="SVM", subset_size=100, corruption_fraction=0.1

The graphs in the following slides represent the training and test accuracies for different models, corruption levels, and k for the digits 1 and 6. A similar pattern occurs for 3 and 8 with the general accuracy lower.

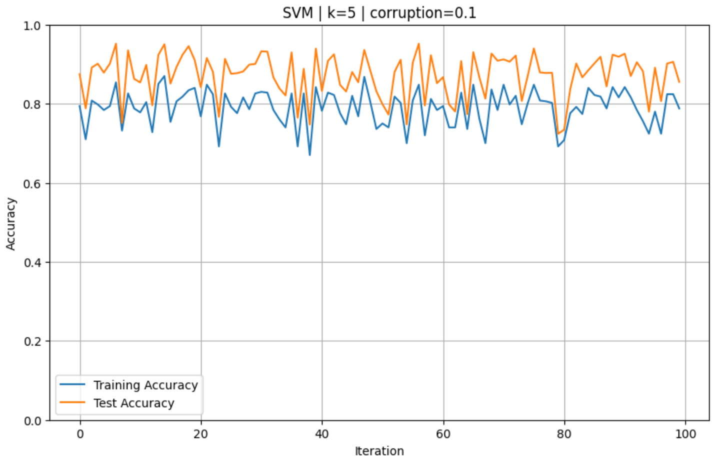
BASELINE PERFORMANCE

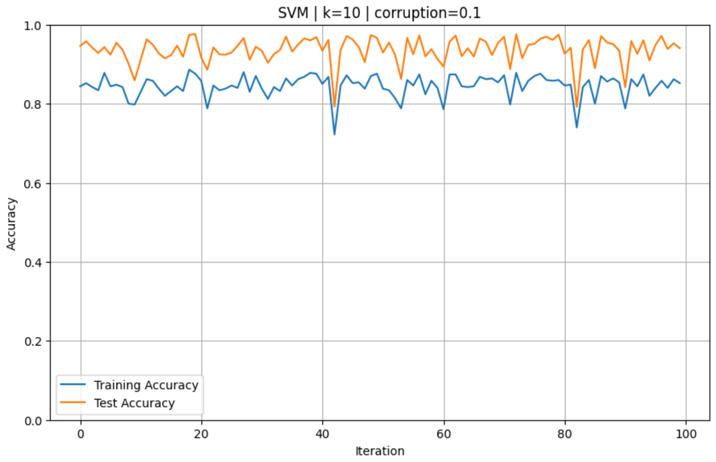
- Objective: Establish a baseline level of accuracy before tuning the hyperparameters and ensemble.
- Models trained on C = 1.0 and used all dimensions.
- The baseline test accuracy for logistic regression = 96%

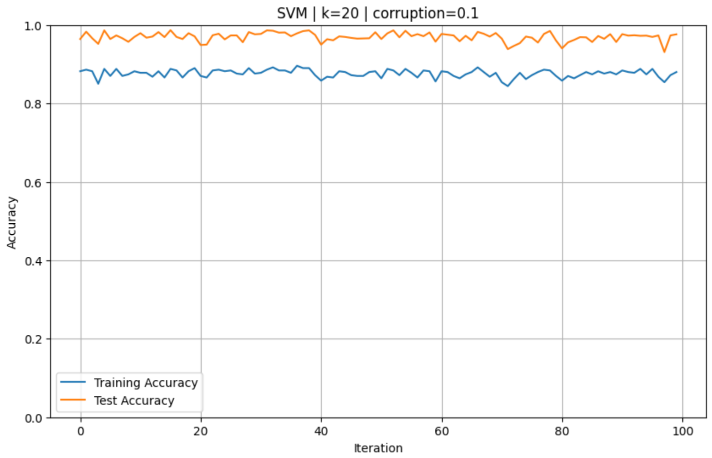


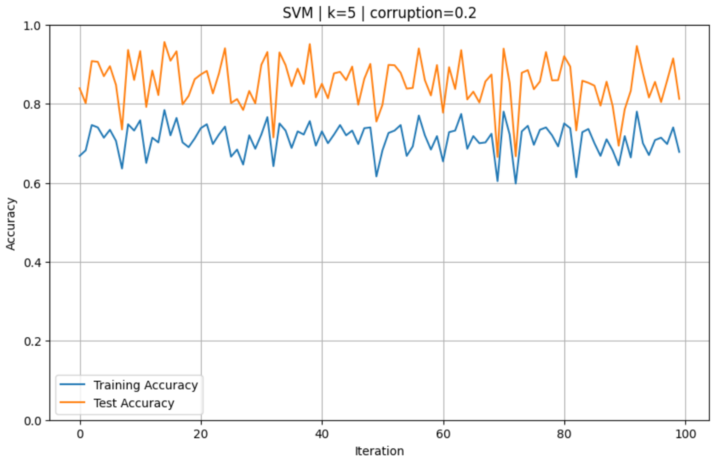


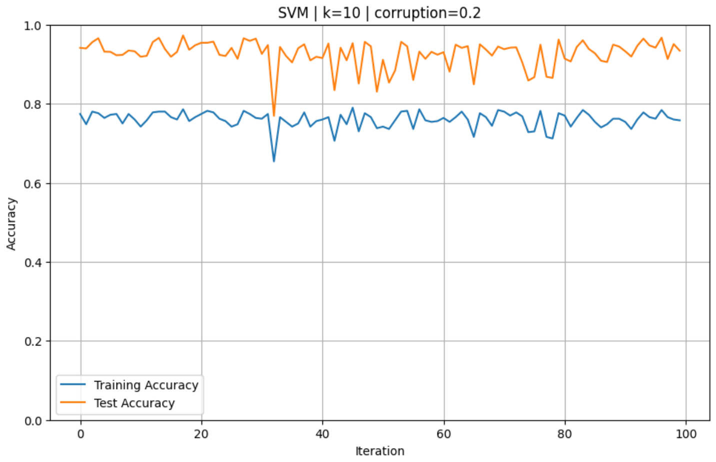
SVM | k=5 | corruption=0.0
SVM | k=10 | corruption=0.0
SVM | k=20 | corruption=0.0
Accuracy
Iteration
Training Accuracy
Test Accuracy

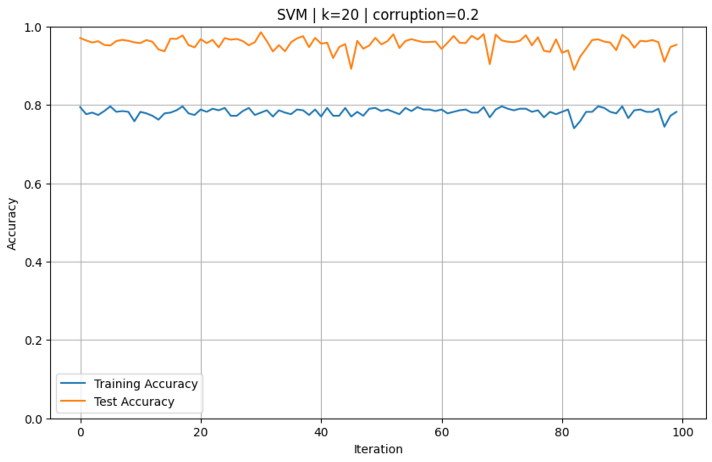

SVM | k=5 | corruption=0.1
Accuracy
Iteration
Training Accuracy
Test Accuracy

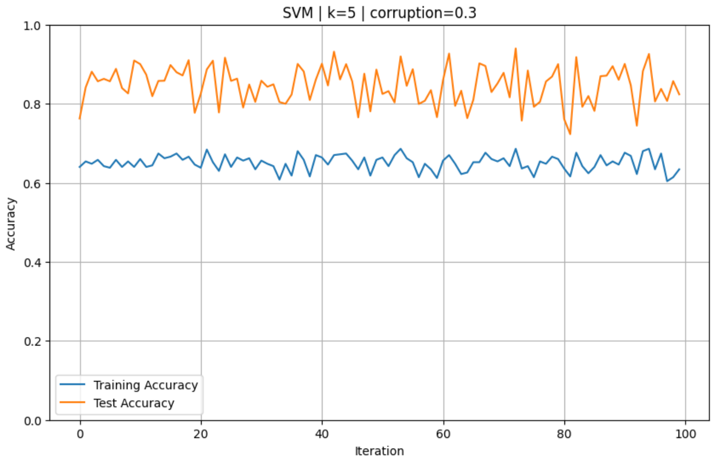

SVM | k=10 | corruption=0.1
Accuracy
Iteration
Training Accuracy
Test Accuracy

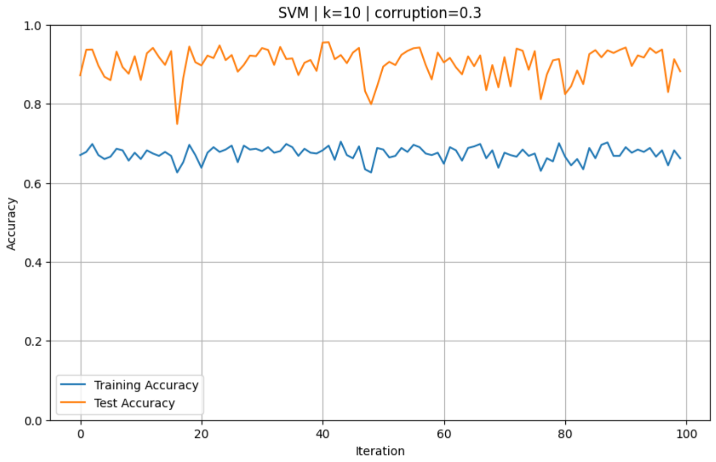

SVM | k=20 | corruption=0.1
Accuracy
Iteration
Training Accuracy
Test Accuracy

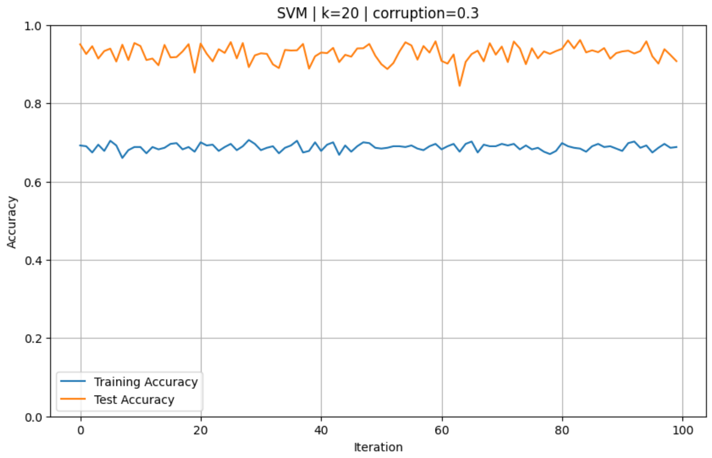

SVM | k=5 | corruption=0.2
Accuracy
Iteration
Training Accuracy
Test Accuracy

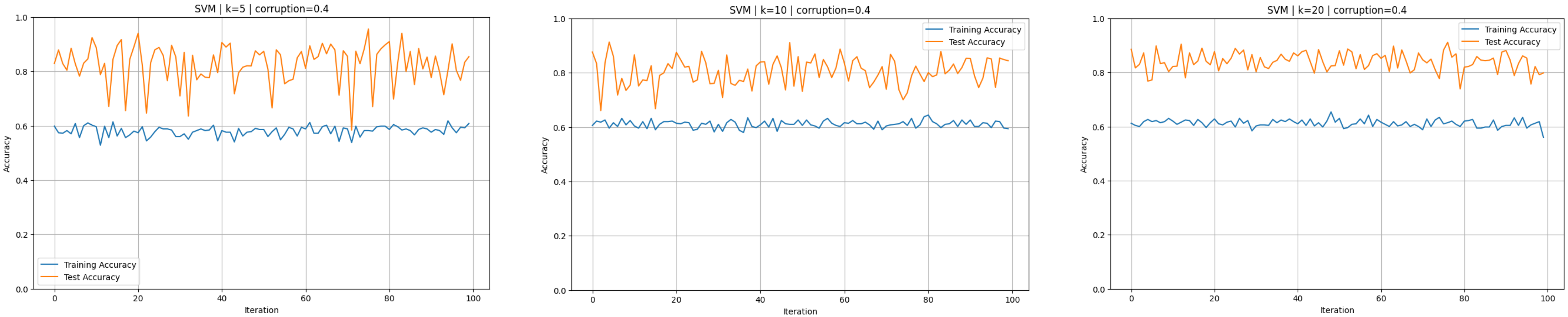

SVM | k=10 | corruption=0.2
Accuracy
Iteration
Training Accuracy
Test Accuracy

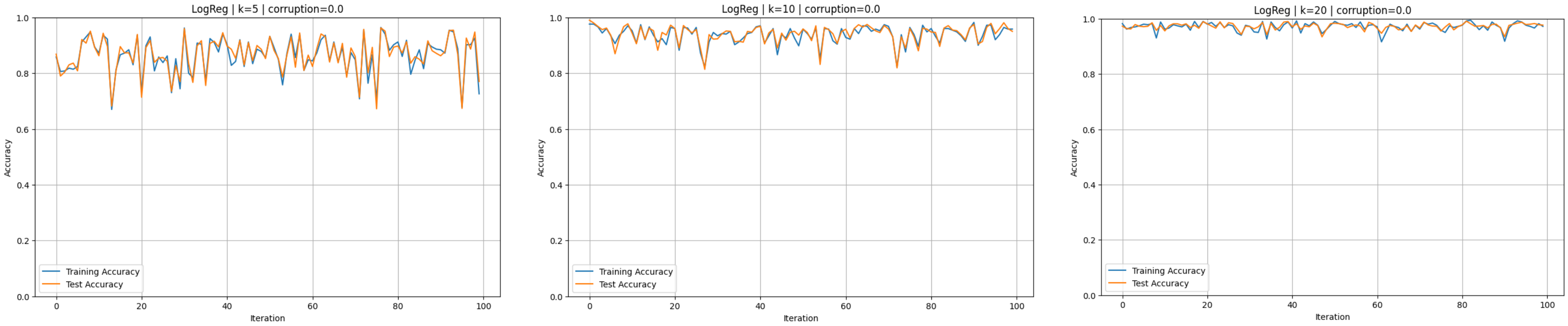

SVM | k=20 | corruption=0.2
Accuracy
Iteration
Training Accuracy
Test Accuracy

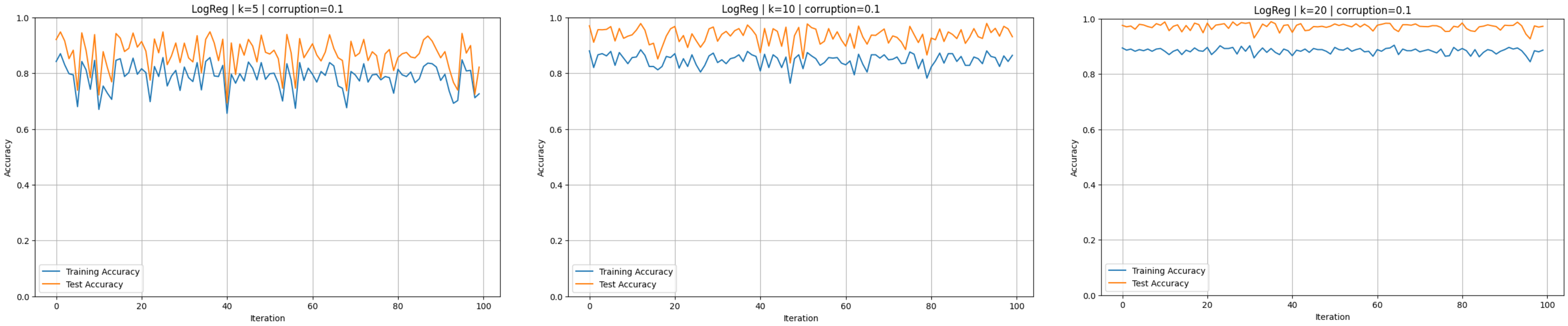

SVM | k=5 | corruption=0.3
Accuracy
Iteration
Training Accuracy
Test Accuracy


SVM | k=10 | corruption=0.3
Accuracy
Iteration
Training Accuracy
Test Accuracy

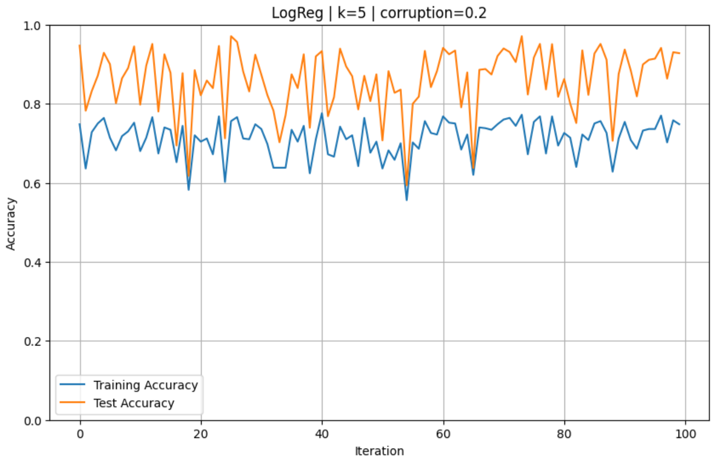
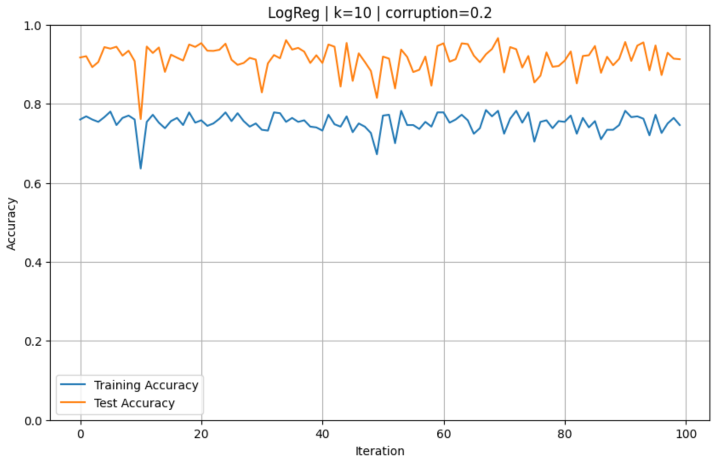
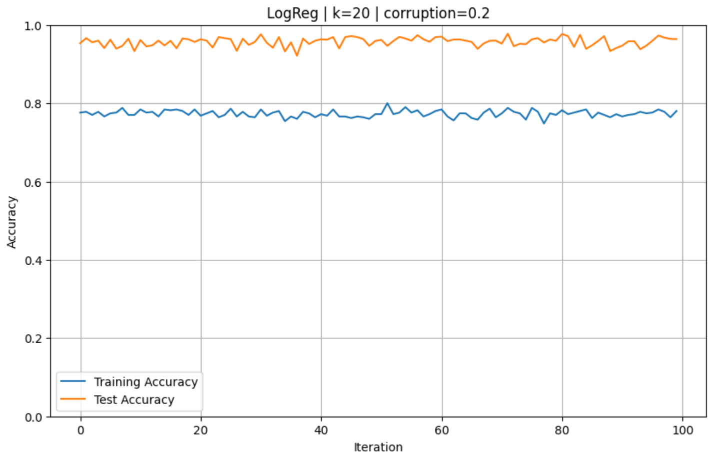

SVM | k=20 | corruption=0.3
Accuracy
Iteration
Training Accuracy
Test Accuracy

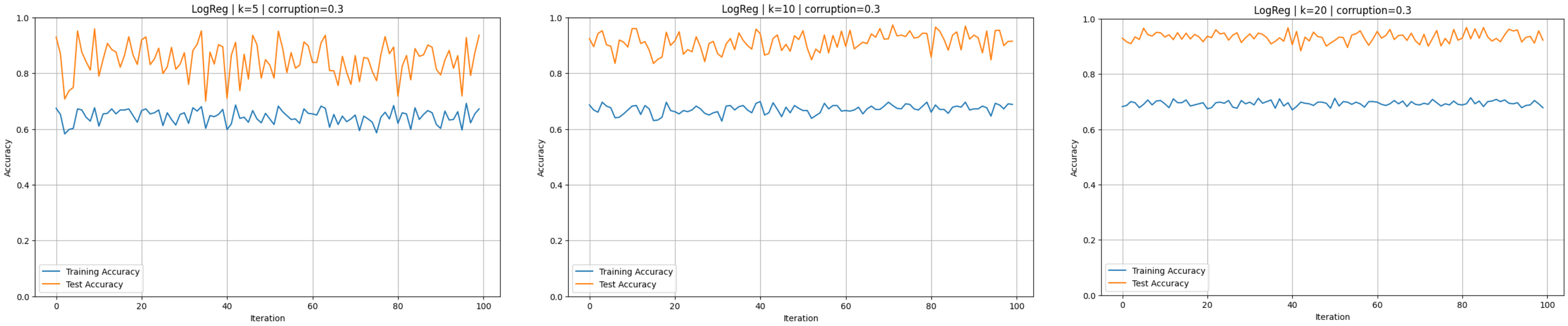

SVM | k=5 | corruption=0.4
Training Accuracy
Test Accuracy
Accuracy
Iteration
SVM | k=10 | corruption=0.4
Training Accuracy
Test Accuracy
Accuracy
Iteration
SVM | k=20 | corruption=0.4
Training Accuracy
Test Accuracy
Accuracy
Iteration

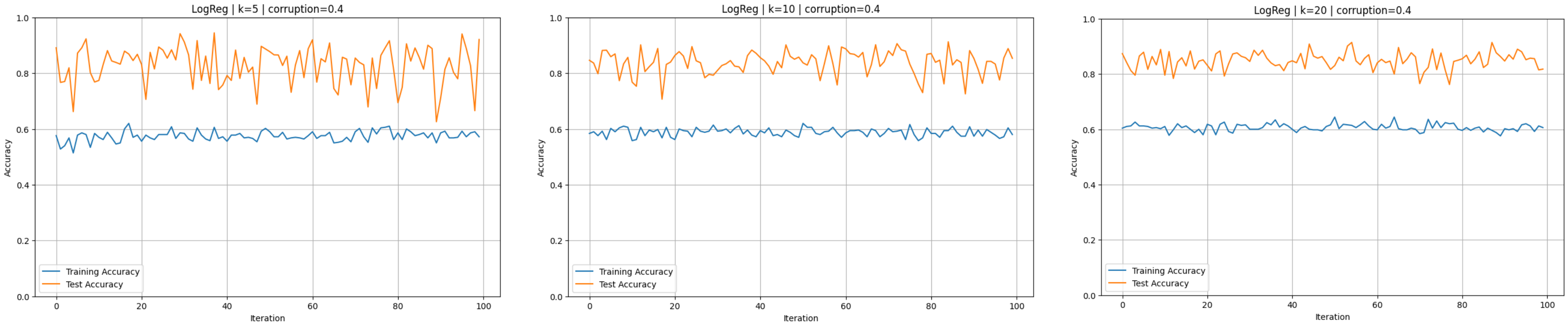

LogReg | k=5 | corruption=0.0
LogReg | k=10 | corruption=0.0
LogReg | k=20 | corruption=0.0
Accuracy
Iteration
Training Accuracy
Test Accuracy
0.0
0.2
0.4
0.6
0.8
1.0
0
20
40
60
80
100



LogReg | k=5 | corruption=0.1
Accuracy
Iteration
Training Accuracy
Test Accuracy
LogReg | k=10 | corruption=0.1
Accuracy
Iteration
Training Accuracy
Test Accuracy
LogReg | k=20 | corruption=0.1
Accuracy
Iteration
Training Accuracy
Test Accuracy

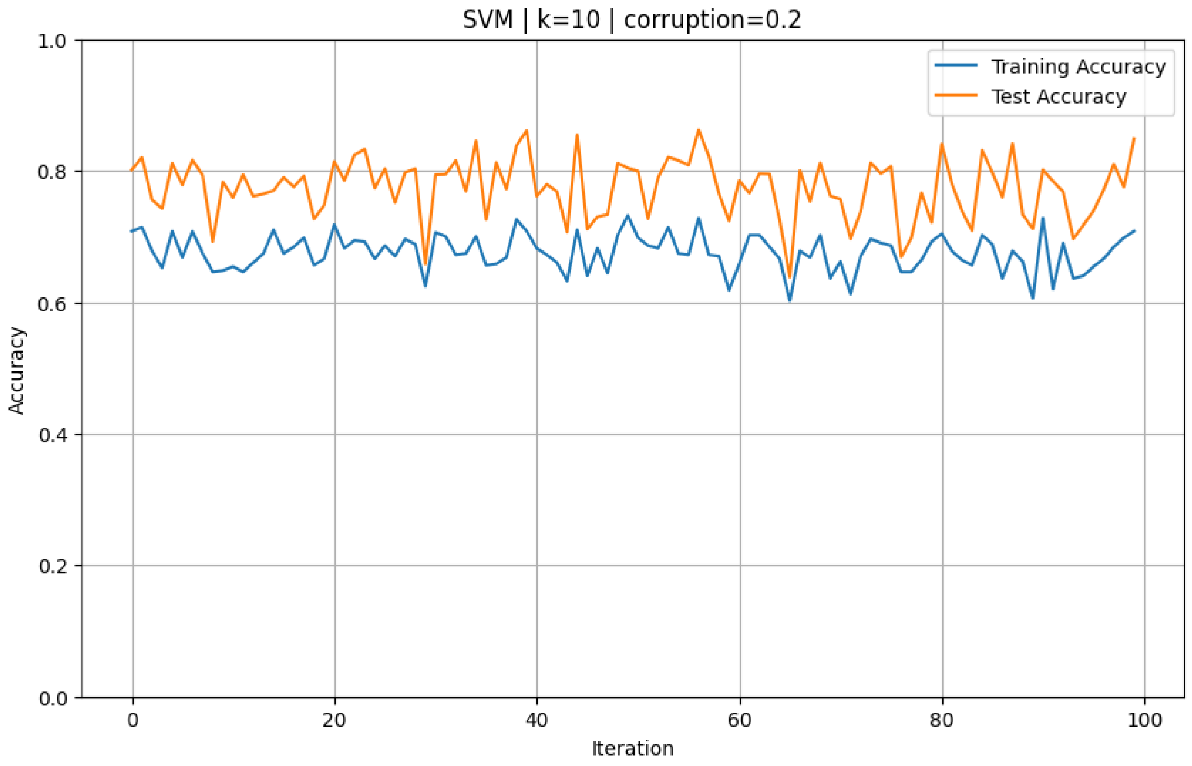

LogReg | k=5 | corruption=0.2
Accuracy
Iteration
Training Accuracy
Test Accuracy

LogReg | k=10 | corruption=0.2
Accuracy
Iteration
Training Accuracy
Test Accuracy

LogReg | k=20 | corruption=0.2
Accuracy
Iteration
Training Accuracy
Test Accuracy

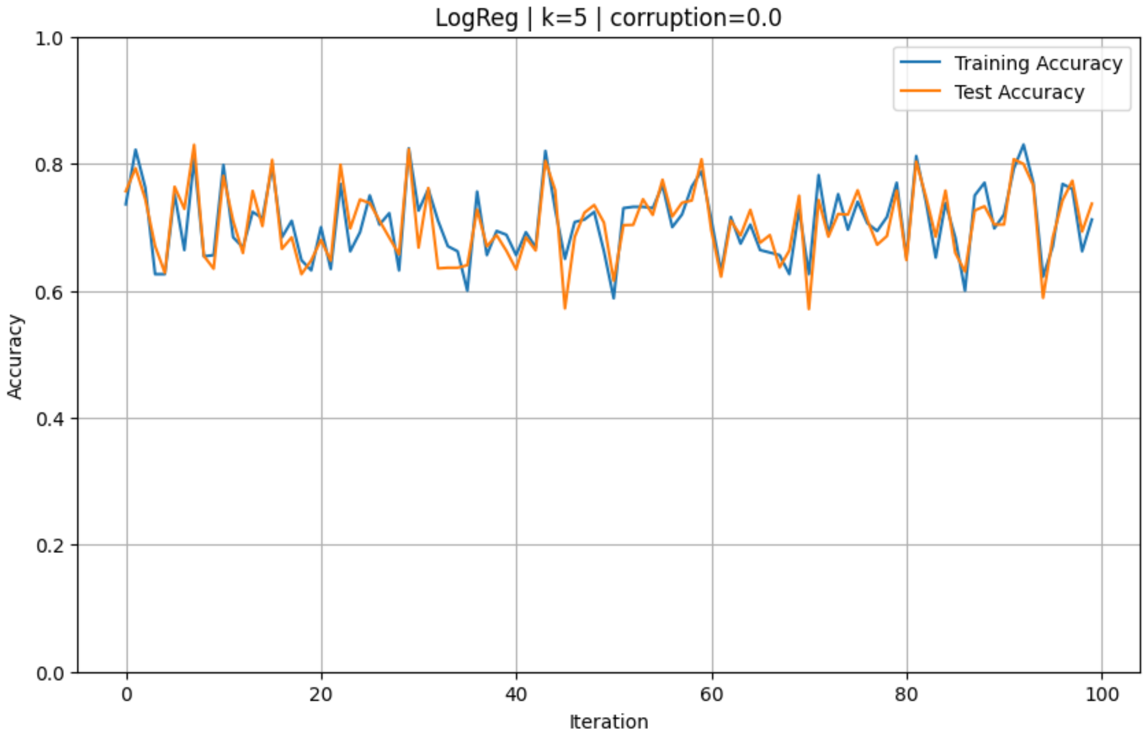

LogReg | k=5 | corruption=0.3
LogReg | k=10 | corruption=0.3
LogReg | k=20 | corruption=0.3
Accuracy
Iteration
Training Accuracy
Test Accuracy
0.0
0.2
0.4
0.6
0.8
1.0
0
20
40
60
80
100

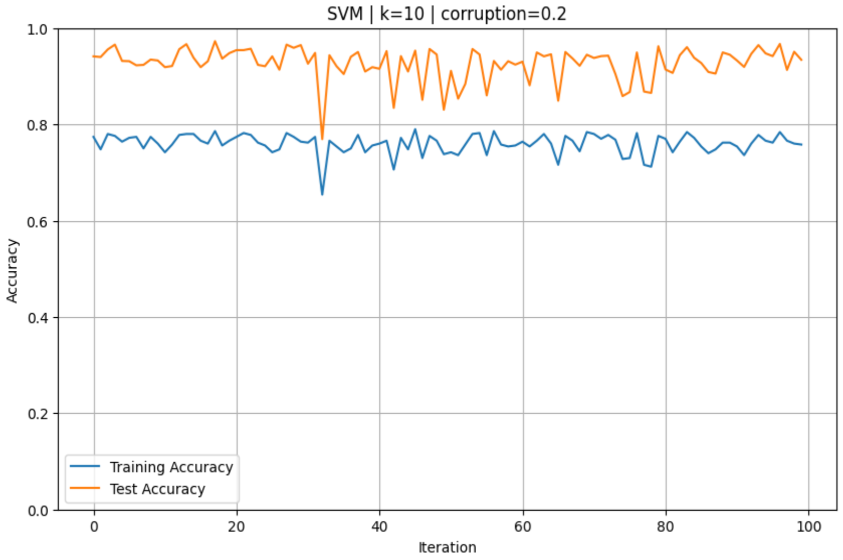


LogReg | k=5 | corruption=0.4
LogReg | k=10 | corruption=0.4
LogReg | k=20 | corruption=0.4
Accuracy
Iteration
Training Accuracy
Test Accuracy

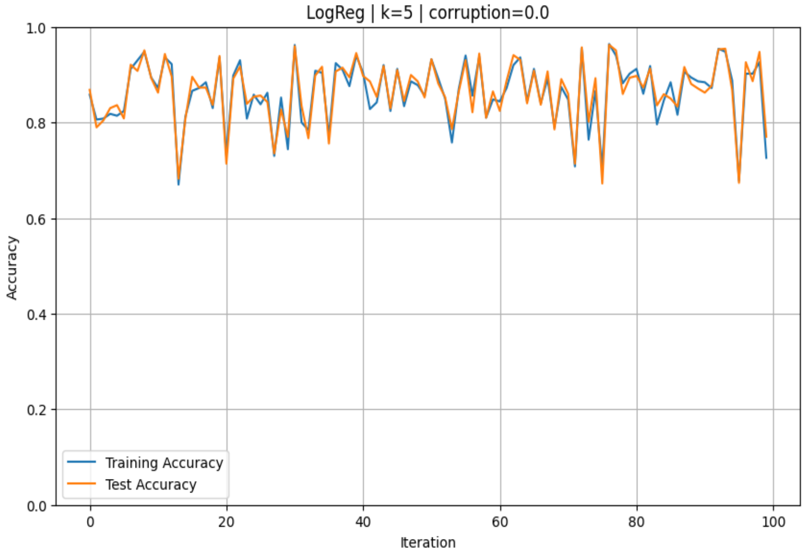
THE SLIDE BELOW SHOWS GRAPGHS FOR 3 VS 8!







- Note the similar patterns



TRAIN –TEST GAP

- The train-test gap equals the training accuracy minus the test accuracy.
- If there is a large positive gap, then the data is performing better on the training set compared to the testing set. (overfitting)
- If there is a large negative gap, then the data is performing better on the testing set compared to the training set.
- When the data has little or no corruption the train and test are similar.

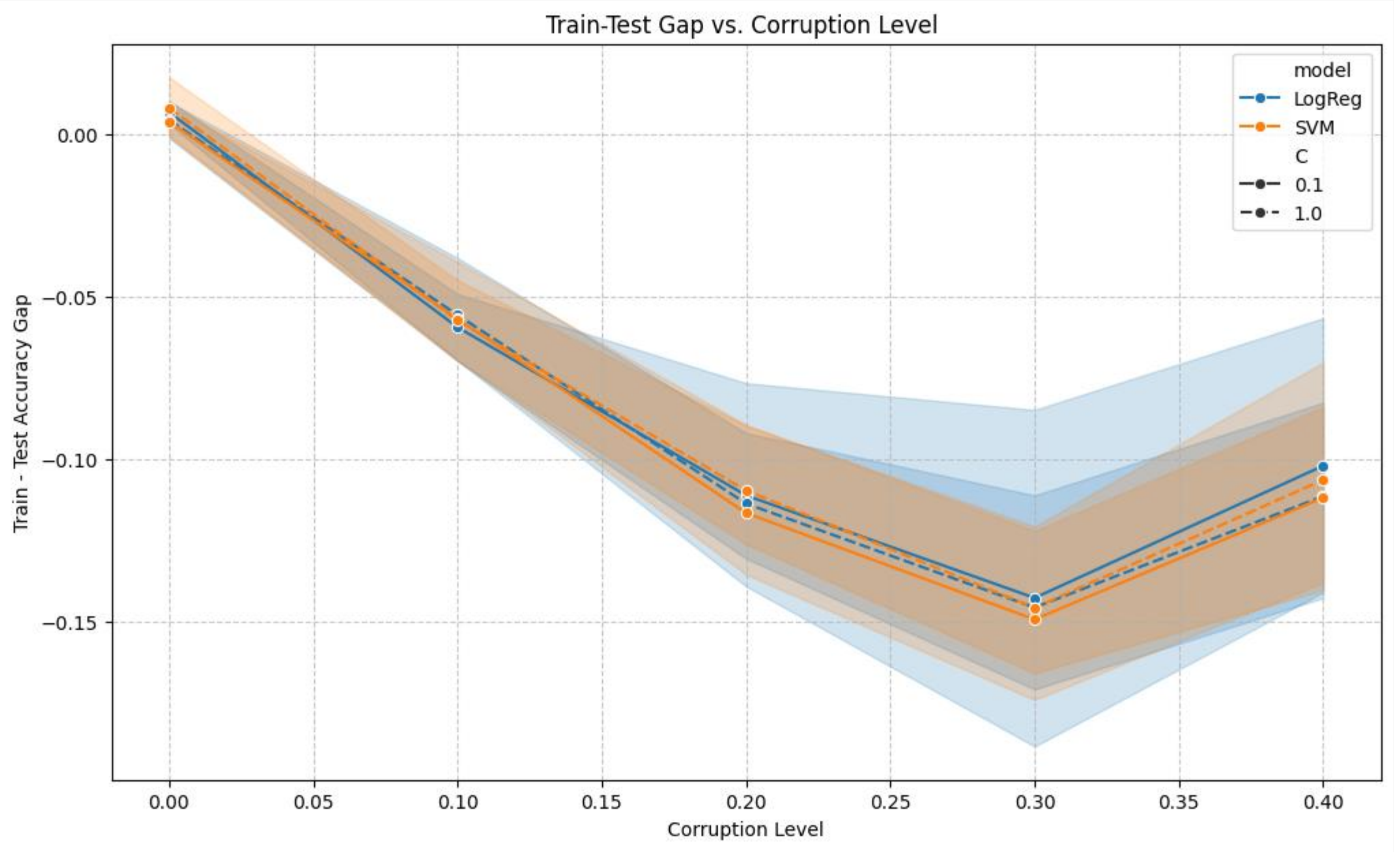
TRAIN –TEST GAP

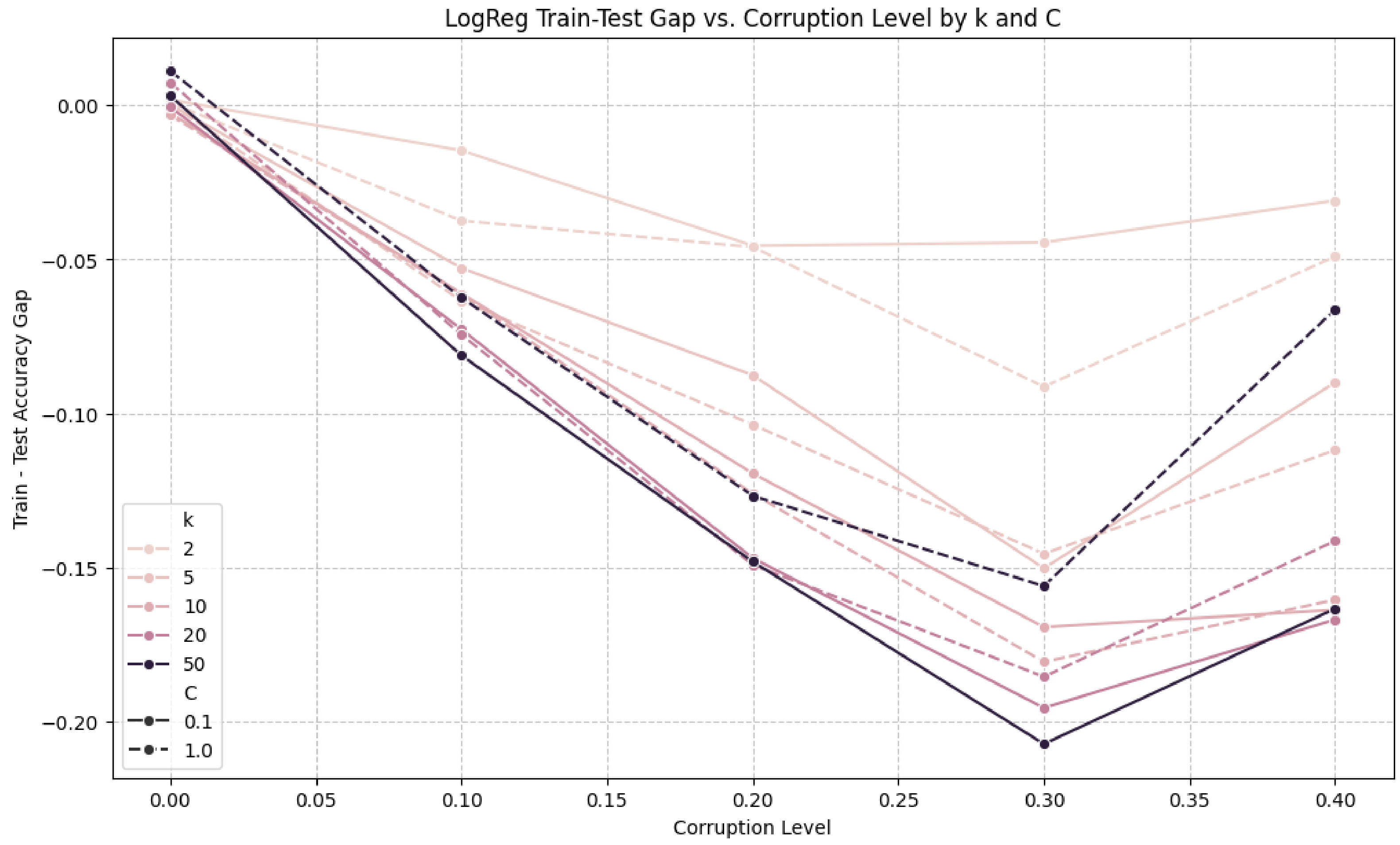
Our Experiment: LogReg with C=0.1 and k=20:

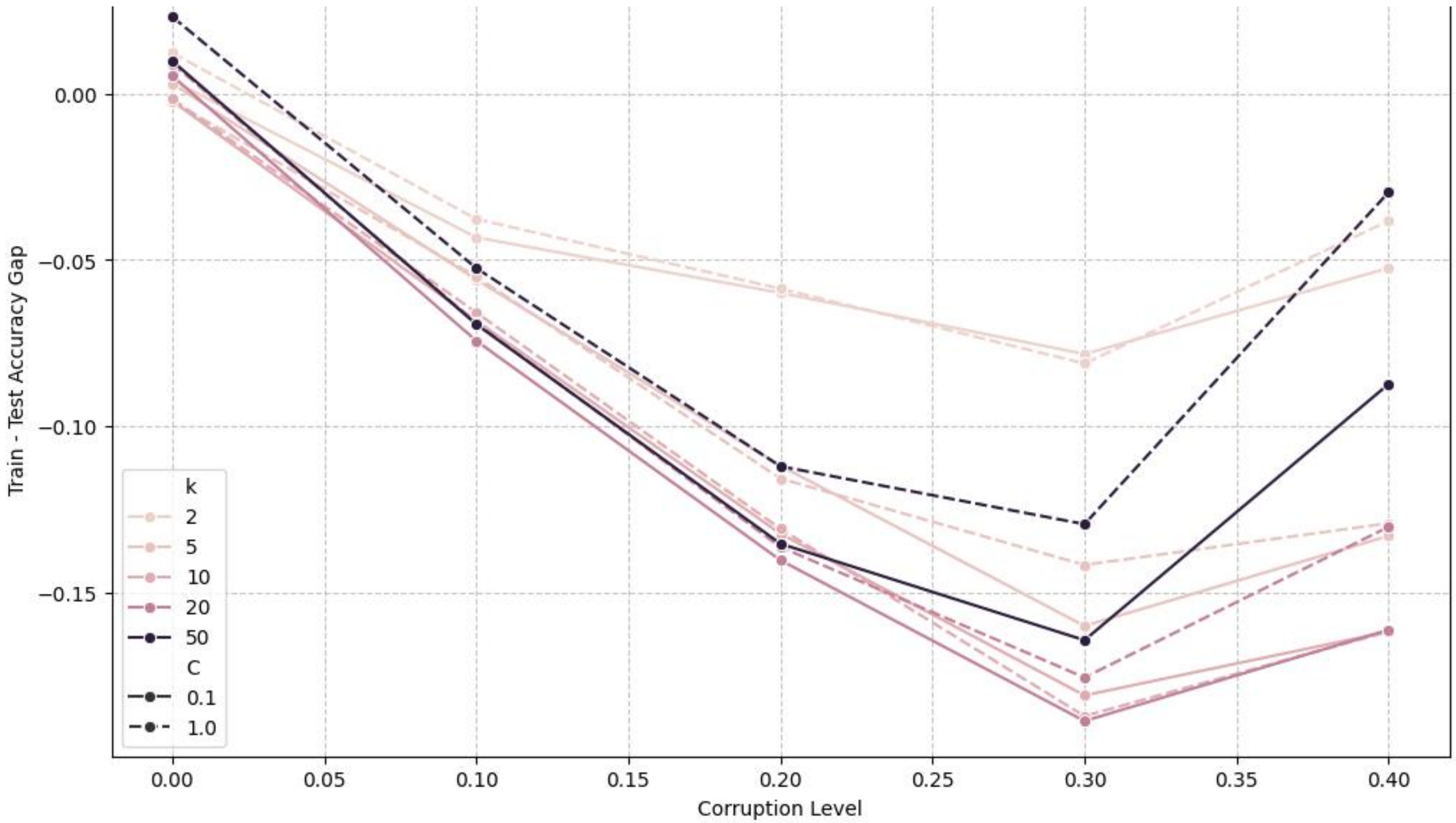
- At 0% corruption there is a very small positive gap: 0.0011 (.8922-.8910)
- At 40% corruption there is a relatively large negative gap: -0.1886 (.6020-.7906)

- What does this negative gap mean?
- The model is performing worse with the training data (corrupted) and better with the testing data (clean).
- Despite being trained on corrupted data the model is still able to perform relatively well on the testing data!
- The model is not overfitting, and it is not fooled by the corruption!
- Why? Perhaps because of the ensemble method and the introduction of C.

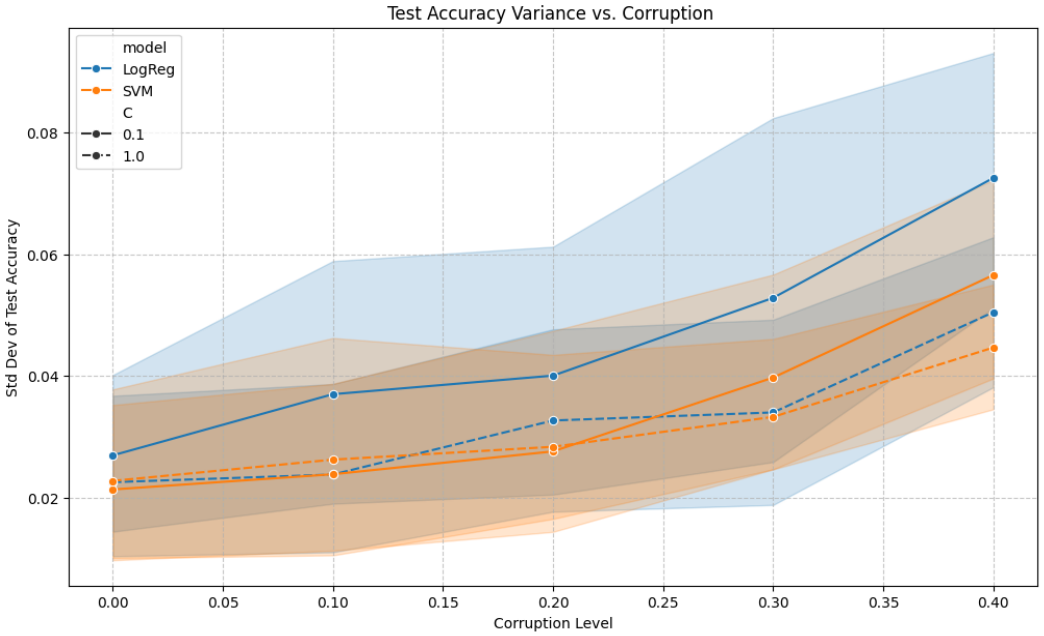
TRAIN –TEST GAP

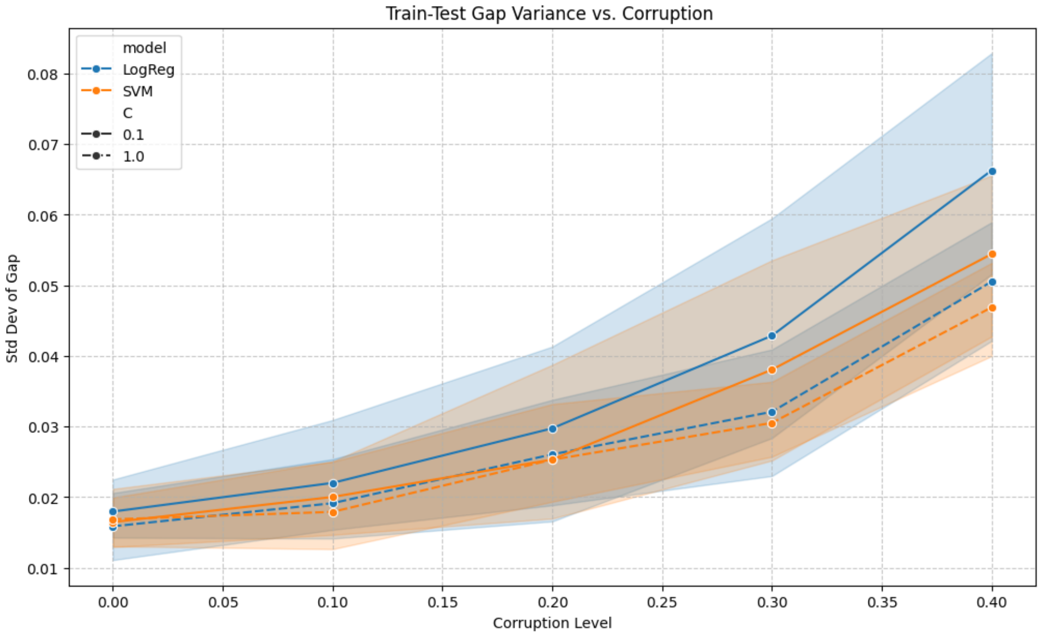



LogReg Train-Test Gap vs. Corruption Level by k and C
Train - Test Accuracy Gap
Corruption Level
0.00
−0.05
−0.10
−0.15
−0.20
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
k
2
5
10
20
50
C
0.1
1.0



Train - Test Accuracy Gap
Corruption Level
0.00
−0.05
−0.10
−0.15
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
k
2
5
10
20
50
C
0.1
1.0

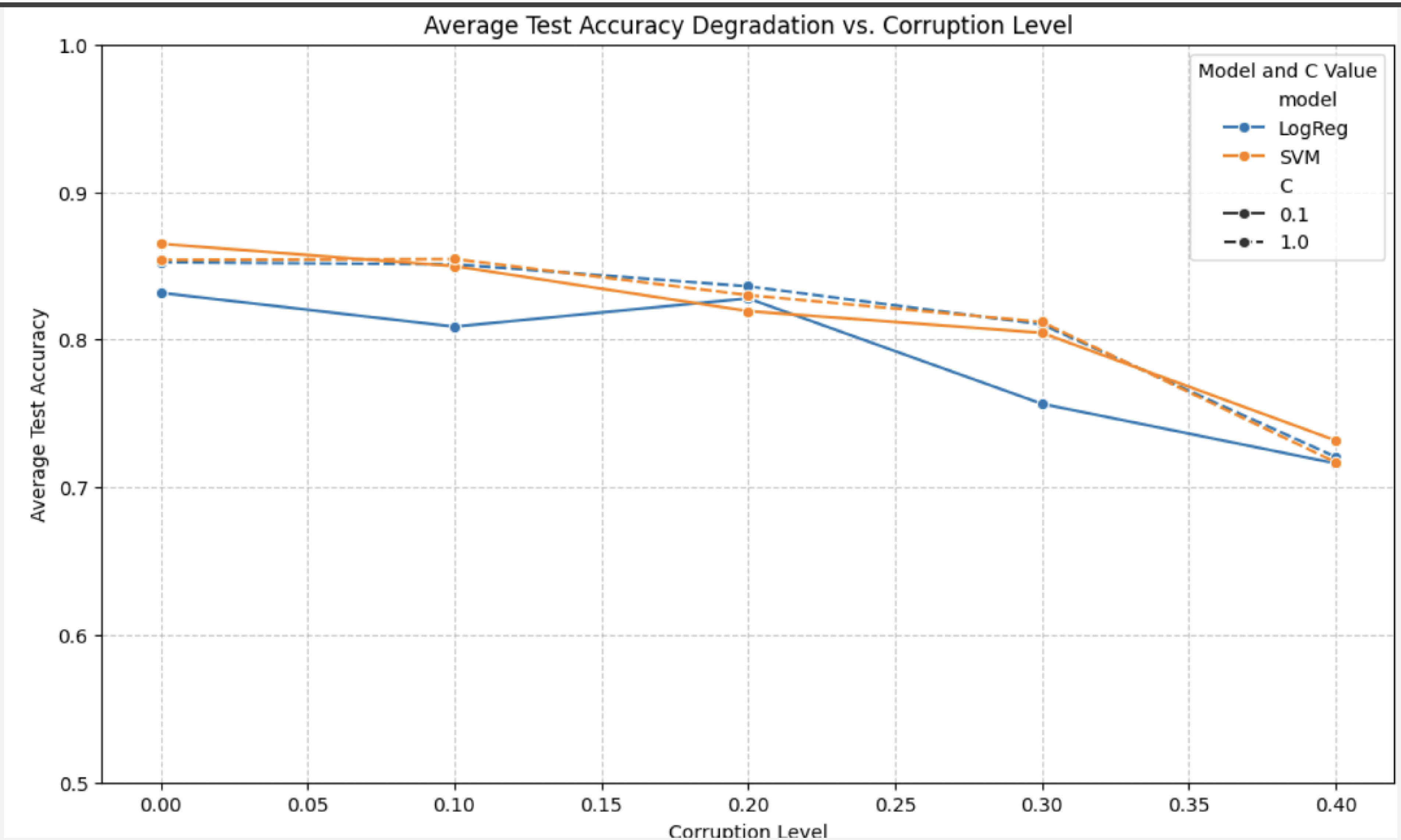




VARIANCE VS. CORRUPTION

WITH INCREASED CORRUPTION WE SEE INCREASE IN VARIANCE → VERY UNSTABLE.

OVERALL PERFORMANCE TRENDS VS CORRUPTION



ROBUSTNESS TO CORRUPTION: IDENTIFYING KEY CONFIGURATIONS

Most Robust Model Configuration (Overall Average Test Accuracy):

Model: SVM
C value: 0.1
kvalue: 50
Overall Average Test Accuracy: 0.8882
Overall Average Test Accuracy Std Dev: 0.0113

Most Robust Model Configuration (Overall Average Test Accuracy): Model: LogReg
C value: 0.1
k value: 50
Overall Average Test Accuracy: 0.8962 / .8917
Overall Average Test Accuracy Std Dev: 0.0133/0.0163

Most Robust Model Configuration at 40% Corruption:

Model: LogReg

C value: 0.1

K value: 20

Mean Test Accuracy: 0.8130

Std Dev Test Accuracy: 0.0263

CONCLUSION & FUTURE WORK

- Logistic Regression vs Linear SVM
- Its relevant! Improving robustness.
- Look at: Other Models, Different Data Sets, Non-random Poisoning

Thank you for listening!

THE ENSEMBLE METHOD

PROBLEM WITH JUST LOGISTIC REGRESSION

Linear regression is a general linear model with the decision hyperplane: $\beta^{T}x+b=0$

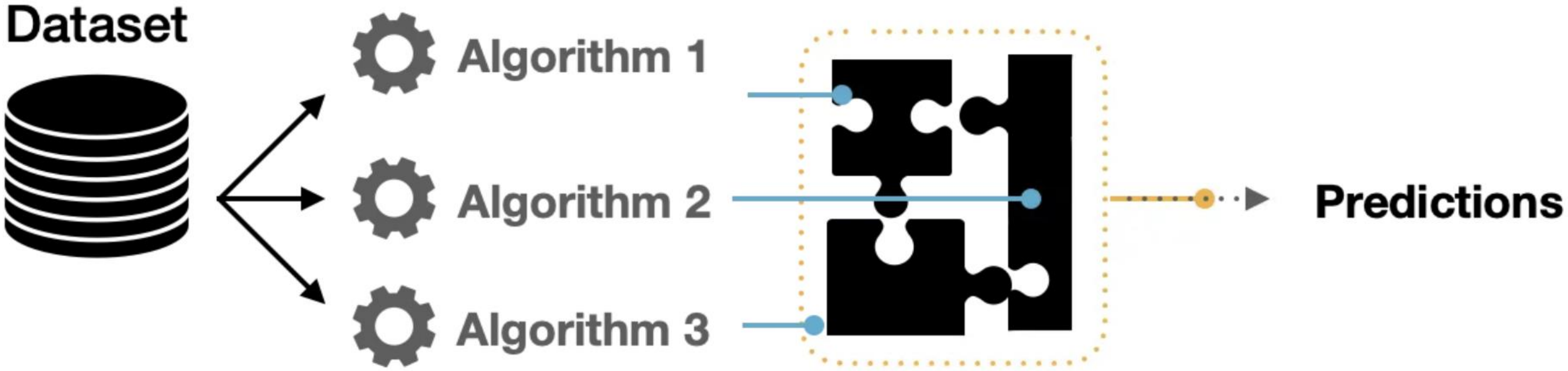
Logistic regression is a linear classifier in the input space. On its own is can only learn one hyper plane.

3 vs 8 is not linearly separable in R^{784}

Ambiguous digits will be misclassified.

WHY ENSEMBLE?

- Single logistic regression: $f(x) = \sigma(\beta^\top x)$
- Ensemble: $F(x) = \sum_{i=1}^{k} w_i \sigma(\beta_i^\top x)$
- Finite mixture of sigmoids -> more expressive
- Convex combination of logistic function
- Combining multiple diverse models (base learners) to solve a prediction problem


Dataset
Algorithm 1
Algorithm 2
Algorithm 3
Predictions

BASIC ENSEMBLE MODELS

$$F_{\text{bag}}(x) = \frac{1}{N}\sum_{i=1}^{N} f_i(x)$$

$$F_{t+1}(x) = F_t(x) + \eta\, h_t(x)$$

$$z_i(x) = f_i(x)$$

$$F_{\text{stack}}(x) = g(z_1(x), z_2(x),\ \dots\ , z_N(x))$$

Bagging (Bootstrap aggragating):

Boosting

Stacking (Stacked Generalization)

THE THREE BASIC ENSEMBLE METHODS ARE **BAGGING** (VARIANCE REDUCTION), **BOOSTING** (BIAS REDUCTION), AND **STACKING** (META-LEARNING TO OPTIMALLY COMBINE MODELS).

"RANDOM SUBSPACE METHOD" ENSEMBLE

- Each model is trained on a number of different randomly generated k dimensional subspaces.
- In our code this is represented by n_models = 5
- For each data point n_models classifies it as either a 0 or a 1.
- If more than half of the n_models classified the point as 1, the final output would be one. Otherwise, it would be 0.
- This technique can reduce overfitting and increase accuracy
- Note: n_models is different than n_runs which represents how many times we repeat the entire ensemble experiment.

RANDOM SUBSPACE PROJECTION (RSP) ENSEMBLE

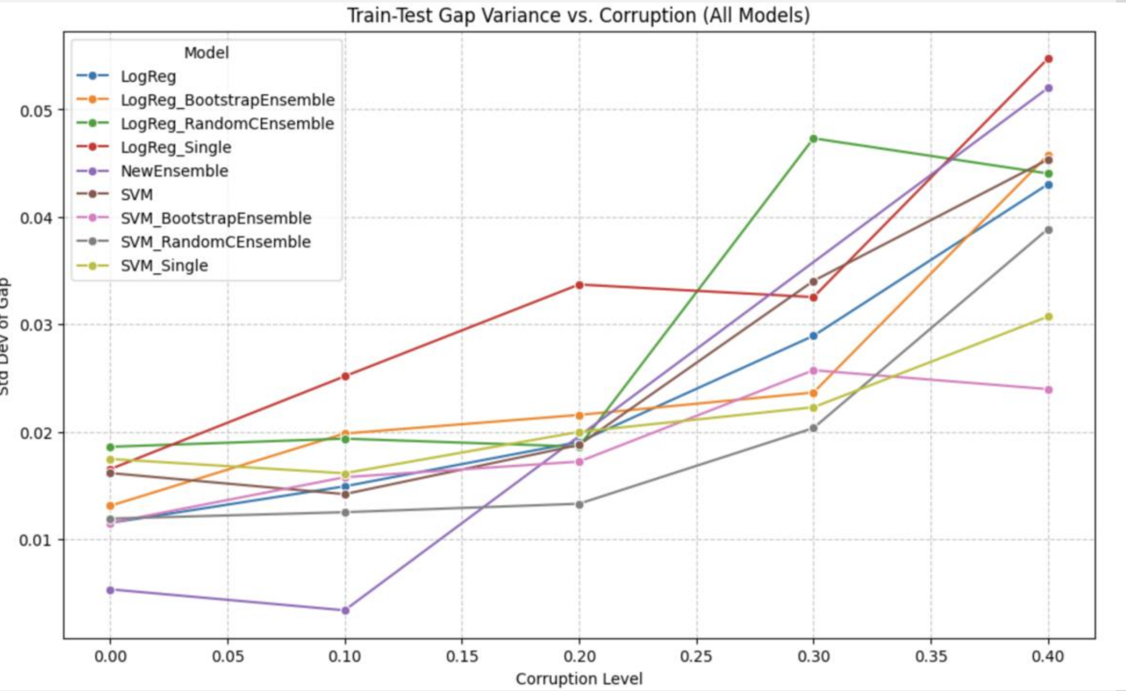
- Instead of training one model on all features, train multiple models, each on a randomly chosen subset (subspace) of features.
- **Steps (The "Hows"):**
  - **Dimensionality Reduction**
  - **Orthogonal Basis**
  - **Individual Base Learners**
  - **Ensemble Aggregation**
- **Why RSP for Robustness?** Diversifies the learning process, averaging out the impact of corrupted labels that might mislead individual classifiers in specific subspaces.

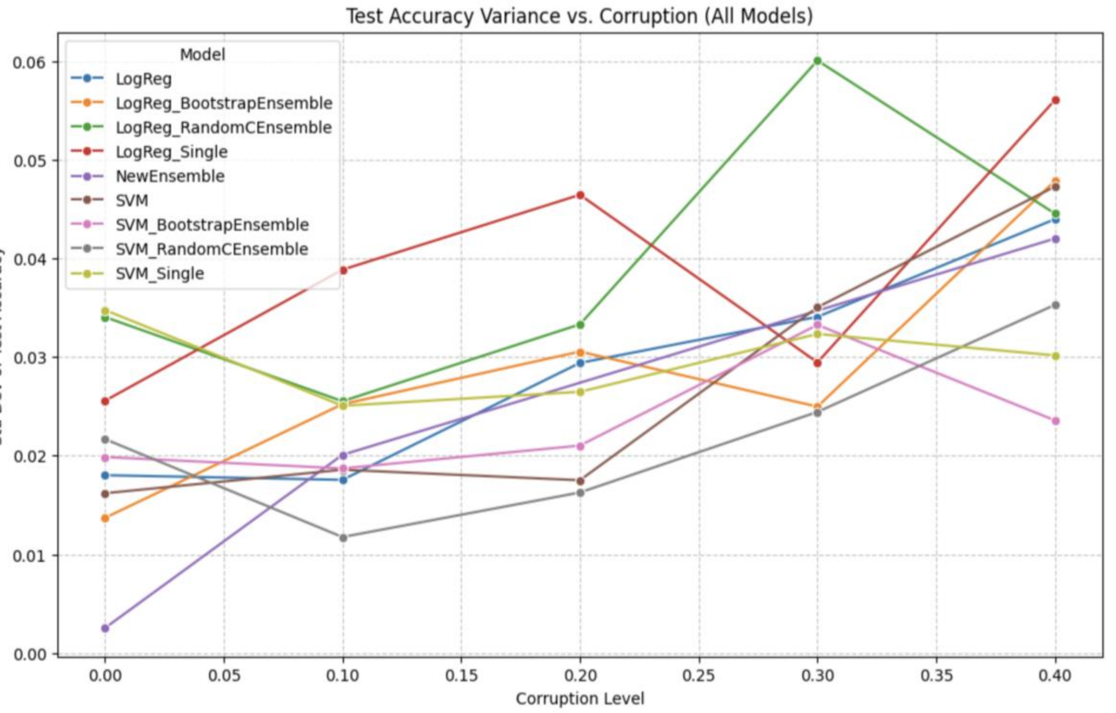
VARIATIONS OF THE ENSEMBLE

- **Standard RSP Ensemble**
- **Bootstrap Ensemble (Bagging variant)**
  - **How:** Samples training data *with replacement* (`data_sampling_method='bootstrap'`) for each base model. This creates even greater diversity in the training sets.
  - **Why:** Further reduces variance by training on slightly different datasets, making the ensemble more stable.
- **Random C Ensemble (Regularization Diversity)**
  - **How:** Randomly selects the regularization parameter C for each base model from a predefined range (`C_values_for_random_sampling`).
  - **Why:** Introduces diversity in model complexity, making the ensemble less prone to being consistently misled by noise if a single C value is suboptimal.
- **Deep 3-Node Ensemble (Custom Architecture)**
  - **How:** A specialized ensemble architecture designed to learn complex decision boundaries, also applied within RSP.
  - **Why:** Explores whether a more sophisticated base learner structure within the ensemble can offer additional robustness.

ENSEMBLE PERFORMANCE: ROBUSTNESS AND IMPACT OF "K"

- **Overall Robustness:** How do different ensembles perform against increasing label corruption?
  - Compare single models vs. various ensemble types across corruption levels.
  - Notice the slower degradation of accuracy and better gap management for ensembles.
- **Impact of Subspace Dimension (k):** How does k affect each ensemble type?
  - For low corruption, higher k often performs better.
  - For high corruption, intermediate k values often provide the best trade-off, balancing information retention and noise filtering.
  - Some ensembles might be more sensitive to k than others.

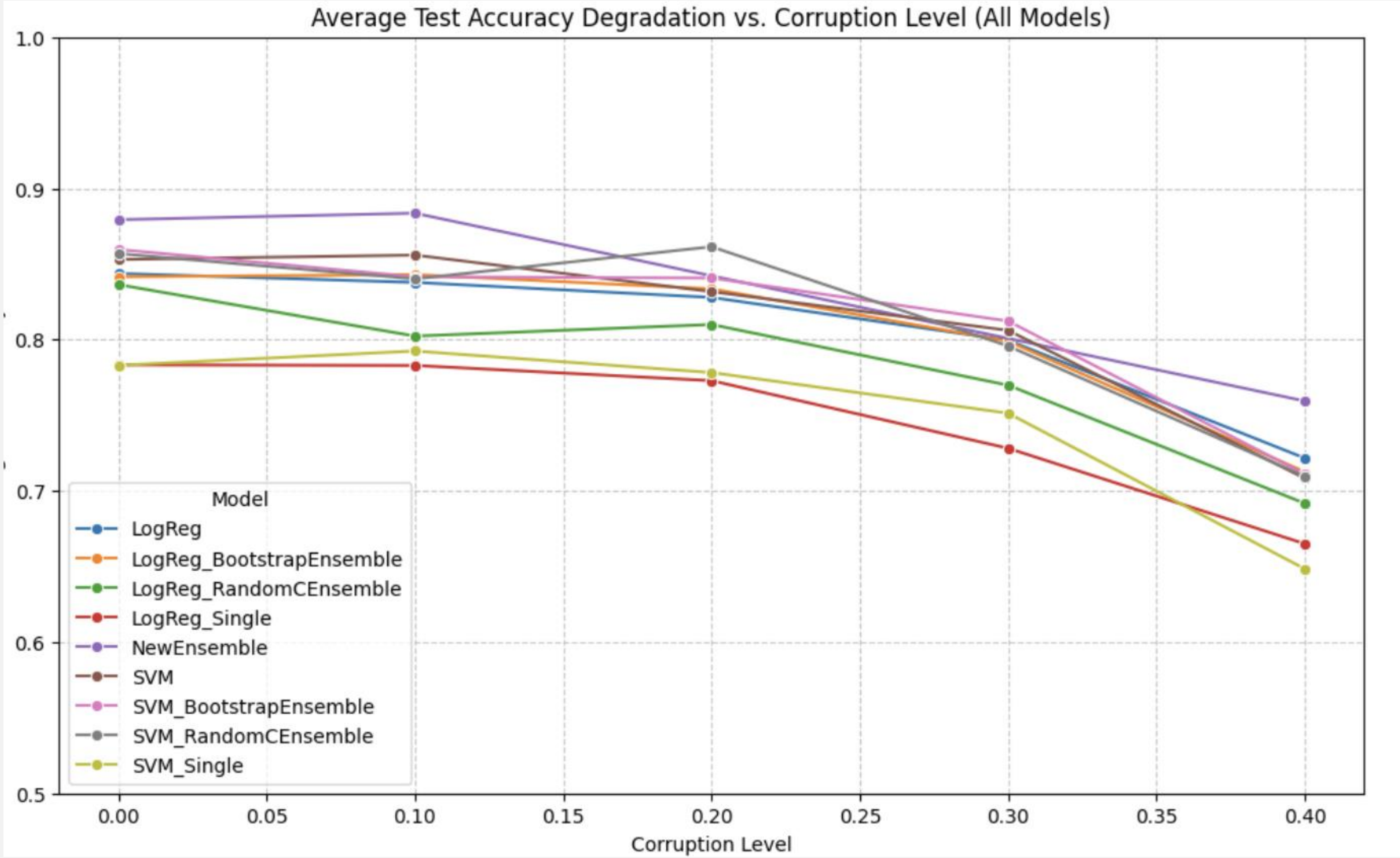




ENSEMBLE PERFORMANCE: STABILITY AND KEY FINDINGS

- **Stability (Variance):** How do ensembles affect the stability of results under attack?
- **Key Findings:**
  - RSP ensembles are effective defensive strategies against label corruption.
  - Optimal k is crucial and corruption-dependent.
  - Different ensemble variations (Bootstrap, Random C, Deep Ensemble) offer diverse ways to enhance robustness.
  - Hyperparameter tuning (e.g., C value) is critical for maximizing ensemble effectiveness.

MODELS TESTED



OUR ENSEMBLE METHOD

- In our code, the **Ensemble** is a "committee" of multiple models working together to make a more reliable prediction than any single model could alone. Specifically, we are using a technique similar to **Random Subspace Method** (an ensemble feature selection approach).
- **1.The "Members" (Diversity)**
- **2. Individual Training**
- **3.The Majority Vote (Aggregation)**
- **Robustness to Corruption:** If one subspace is heavily affected by the label noise, the other four models can "outvote" it and correct the error.

MATH BEHIND OUR ENSEMBLE METHOD

- **1. Variance Reduction via Averaging (Law of Large Numbers) -> shrinking error bars**
- **2. The Condorcet Jury Theorem -> majority vote curve**
- **3. Dimensionality and Johnson-Lindenstrauss (JL) Lemma -> multiple projections of MNIST**
- **The Math:** By using multiple random projections and ensembling them, we are essentially sampling multiple "distance-preserving" views of the data.
- Where one projection might accidentally collapse two different digits (like a 3 and an 8) into the same space, it is mathematically highly improbable that *all* random projections will make that same mistake.
- **4. Robustness to Label Noise (Corruption) -> smoothing corrupted labels**
- If the labels are corrupted by a rate ϵ, the corruption acts as **stochastic noise**. In a single model, this noise biases the decision boundary. In an ensemble, the averaging process acts as a **low-pass filter**, filtering out the "high-frequency" noise (the random corrupted labels) and keeping the "low-frequency" signal (the actual structure of the MNIST digits).

VARIANCE REDUCTION VIA AVERAGING

- Each logistic regression model trained on corrupted data or randomly projected subspace has high variance.
- Averaging N models reduces the variance by a factor of N
- Averaging cancels noise but preserves signals

$$\bar{X} = \frac{1}{N}\sum_{i=1}^{N} X_i$$

$$\mathrm{Var}(\bar{X}) = \frac{\sigma^2}{N}$$

CONDORCET JURY THEOREM

Let X_i be the indicator that model i predicts correctly:

$$P(X_i = 1) = p > \frac{1}{2}$$

Let $S_N = \sum_{i=1}^{N} X_i$.
The ensemble is correct if:

$$S_N > \frac{N}{2}$$

Then:

$$P\left(S_N > \frac{N}{2}\right) \underset{N \to \infty}{\to} 1$$

- If each model has prob p > 0.5 of being correct, then the probability of the majority vote being correct increases with the number of models

JOHNSON–LINDENSTRAUSS LEMMA

For any $0 < \epsilon < 1$, and any set of m points in R^d, there exists a mapping $f: R^d \to R^k$ such that:

$$(1-\epsilon)\|u-v\|^2 \le \|f(u)-f(v)\|^2 \le (1+\epsilon)\|u-v\|^2$$

provided:

$$k = O\left(\frac{\log m}{\epsilon^2}\right)$$

- high-dimensional data can be projected into a lower-dimensional space while approximately preserving pairwise distances

ROBUSTNESS TO LABEL NOISE

Let the true label be Y.

Let the corrupted label be:

$$\tilde{Y} = \begin{cases} Y & \text{with probability } 1 - \epsilon \\ 1 - Y & \text{with probability } \epsilon \end{cases}$$

A single model learns:

$$P(\tilde{Y} = 1 \mid x) = (1 - \epsilon)P(Y = 1 \mid x) + \epsilon(1 - P(Y = 1 \mid x))$$

This biases the decision boundary.

But the ensemble prediction:

$$\hat{p}(x) = \frac{1}{N}\sum_{i=1}^{N} p_i(x)$$

converges to the **expected uncorrupted signal** as N grows.

- If labels are corrupted at rate ϵ, each model is trained on a noisy version of the truth. Averaging acts as a low-pass filter, removing high-frequency noise.

REFERENCES

- https://www.jmlr.org/papers/volume22/20-600/20-600.pdf